ParkPulse: full build prompt

Paste everything below the line into an AI coding assistant (Claude Code, Cursor, etc.). It describes the exact prototype: a crowdsourced parking app with a live shared map, sign reader, push alerts and points.

---

1. Role and goal

You are a senior Flutter and Firebase engineer. Build a complete, compiling prototype called **ParkPulse**: a mobile app where drivers see open street parking spots reported by other users in real time, read parking rules in plain English, scan a parking sign with the camera, get push alerts when a spot opens near their destination, and earn points. Create every file in full. Do not leave TODOs or placeholders, except the values the user must supply (Firebase config, server URL, optional Mapbox token). After writing code, review it line by line for compile errors, then give the exact commands to run it.

2. Tech stack

- **App:** Flutter (stable, Material 3), Dart 3, Android first, iOS compatible.
- **Packages:** `firebase_core`, `firebase_auth`, `cloud_firestore`, `firebase_messaging`, `flutter_map` (v7 API), `latlong2`, `geolocator`, `image_picker`, `http`. Also `dart:convert`.
- **Map:** OpenStreetMap tiles `https://tile.openstreetmap.org/{z}/{x}/{y}.png`, `userAgentPackageName: 'com.example.parkpulse'`. A constant `kMapboxToken = ''`; if non-empty use `https://api.mapbox.com/styles/v1/mapbox/streets-v12/tiles/256/{z}/{x}/{y}@2x?access_token=$kMapboxToken` instead.
- **Backend:** Firebase Anonymous Auth, Cloud Firestore, Cloud Functions v2 (Node 20, JavaScript, `firebase-admin` ^12, `firebase-functions` ^5), Firebase Cloud Messaging.
- **Sign reader server:** Python, FastAPI, Uvicorn, EasyOCR, Pillow, NumPy, optional Ultralytics YOLOv8.
- **Geocoding:** Nominatim (OpenStreetMap) for search and reverse geocoding, with header `User-Agent: ParkPulse/1.0`.

3. Project structure

```
parkpulse_flutter/
  README.md
  firestore.rules
  lib/main.dart            (single file app)
  functions/index.js
  functions/package.json
  sign_ai/server.py
  sign_ai/decode.py
  sign_ai/train.py
  sign_ai/data.yaml
  sign_ai/requirements.txt
  sign_ai/README.md
```

`firebase_options.dart` is generated by `flutterfire configure` and imported as `import 'firebase_options.dart';`.

4. Setup steps the README must contain

1. `flutter create parkpulse && cd parkpulse`, then replace `lib/main.dart`.
2. `flutter pub add firebase_core firebase_auth cloud_firestore firebase_messaging flutter_map latlong2 geolocator image_picker http`.
3. Create a Firebase project. Enable Authentication > Anonymous, create Firestore, paste `firestore.rules`.
4. `dart pub global activate flutterfire_cli`, then `flutterfire configure`.
5. Android manifest permissions: `INTERNET`, `ACCESS_FINE_LOCATION`, `POST_NOTIFICATIONS`, and camera. For local http testing add `android:usesCleartextTraffic="true"`. iOS: `NSLocationWhenInUseUsageDescription`, `NSCameraUsageDescription`, `NSPhotoLibraryUsageDescription`.
6. Backend: install Firebase CLI, `firebase login`, `firebase init` (Firestore and Functions, JavaScript), copy `functions/index.js`, `functions/package.json` and `firestore.rules`, then `firebase deploy --only functions,firestore:rules`. Mention Cloud Functions need the Blaze plan (free monthly allowance).
7. Sign server: `pip install -r requirements.txt`, `uvicorn server:app --host 0.0.0.0 --port 8000`, then set `kSignApi` in `lib/main.dart`.

8. `flutter run` on a real phone.

The README must also list what is not included: trained YOLO model, street-segment rules from city open data, photo upload on reports, rate limiting, and a production map tile provider.

5. Firestore data model

All documents are created by signed-in (anonymous) users.

Collection	Fields	Notes
`spots/{id}`	`lat` (number), `lng` (number), `uid` (string), `city` (string), `note` (string), `taken` (bool, starts false), `createdAt` (server timestamp), `takenBy` (string, set when taken)	An open street spot. Shown for 30 minutes.
`garages/{id}`	`lat`, `lng`, `uid`, `city`, `name` (string), `price` (number per hour), `createdAt`	A parking garage.
`rules/{id}`	`lat`, `lng`, `uid`, `type` (`free`, `paid` or `no`), `text` (string under 200 chars), `createdAt`	A sign or rule pinned on the map.
`users/{uid}`	`points` (number)	Read only for the client. Written only by Cloud Functions.
`alerts/{uid}`	`token` (FCM token), `alertOn` (bool), `destLat`, `destLng`	Written by the client. Read by Cloud Functions.

6. Security rules (`firestore.rules`)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{db}/documents {
    match /spots/{id} {
      allow read: if request.auth != null;
      allow create: if request.auth != null
        && request.resource.data.uid == request.auth.uid
```

```
        &&
request.resource.data.keys().hasOnly(['lat','lng','uid','createdAt','taken','
note','city'])
        && request.resource.data.createdAt == request.time;
      allow update: if request.auth != null
        && resource.data.taken == false
        &&
request.resource.data.diff(resource.data).affectedKeys().hasOnly(['taken','ta
kenBy'])
        && request.resource.data.takenBy == request.auth.uid;
    }
    match /garages/{id} {
      allow read: if request.auth != null;
      allow create: if request.auth != null
        && request.resource.data.uid == request.auth.uid
        &&
request.resource.data.keys().hasOnly(['lat','lng','uid','city','createdAt','n
ame','price'])
        && request.resource.data.name.size() < 80;
    }
    match /rules/{id} {
      allow read: if request.auth != null;
      allow create: if request.auth != null
        && request.resource.data.uid == request.auth.uid
        &&
request.resource.data.keys().hasOnly(['lat','lng','uid','type','text','create
dAt'])
        && request.resource.data.text.size() < 200;
    }
    match /users/{u} {
      allow read: if request.auth != null && request.auth.uid == u;
    }
    match /alerts/{u} {
      allow read, write: if request.auth != null && request.auth.uid == u
        &&
request.resource.data.keys().hasOnly(['token','alertOn','destLat','destLng'])
;
    }
  }
}
```

7. Cloud Functions (`functions/index.js`)

Use `onDocumentCreated` and `onDocumentUpdated` from `firebase-functions/v2/firestore`, plus `firebase-admin` Firestore and Messaging. Helper `pts(uid, n)` does `set({points: FieldValue.increment(n)}, {merge: true})` on `users/{uid}`. Helper `km(lat1, lng1, lat2, lng2)` is the haversine distance in kilometres.

- **`onSpot`** (spot created): give the reporter +10 points. Query `alerts` where `destLat` is greater than `lat - 0.5/111` and less than `lat + 0.5/111` (range on one field only, so no composite index). In code keep documents where `alertOn` is true, a `token` exists, the document id is not the reporter, and `km(...) < 0.5`. Send `getMessaging().sendEachForMulticast` with title "Parking spot opened", body "A spot just opened near your destination.", Android priority high.
- **`onRule`** (rule created): +5 points to the author.
- **`onGarage`** (garage created): +10 points to the author.
- **`onTaken`** (spot updated): if `taken` changed from false to true, `takenBy` is set and `takenBy` is not the reporter, +3 points to `takenBy`.

`package.json`: name `functions`, main `index.js`, engines node 20, the two dependencies above.

8. Flutter app specification (`lib/main.dart`)

8.1 Startup and theme

`main()` initializes Firebase with `DefaultFirebaseOptions.currentPlatform`, signs in anonymously if there is no current user, then runs `ParkApp`. `MaterialApp`: title "ParkPulse", no debug banner, Material 3, `colorSchemeSeed: Color(0xFF0A4296)`. Home is `MapPage`. Constants at top: `kMapboxToken = ''` and `kSignApi = 'http://YOUR-SERVER:8000'`. Define `typedef Doc = QueryDocumentSnapshot<Map<String, dynamic>>` and a helper `_ll(Map d)` returning `LatLng` from `lat` and `lng` (cast through `num`).

8.2 State

Fields: Firestore instance, current uid, `MapController`, `LatLng? _dest`, `bool _pickDest`, `bool _first` (true until the first spots snapshot), `String? _filter` (null, `free`, `paid` or `garage`), a search `TextEditingController`. Streams, created once (not

inside build): `_spots` = `spots` where `createdAt` is greater than now minus 30 minutes, ordered by `createdAt` descending, limit 200; `_rulesS` = `rules` limit 300; `_garS` = `garages` limit 300.

8.3 Map layer

`FlutterMap` fills the whole screen, initial centre `LatLng(29.03, 77.07)` (near Murthal), zoom 15. On start, get the GPS location and move the map to it at zoom 16, ignoring errors. A single-tap sets the destination only when `_pickDest` is true. A long-press anywhere opens the add-rule dialog at that point. Children in order:

1. Tile layer (OpenStreetMap or Mapbox as in section 2).
2. **Rule markers** (34 by 34 circles). Show a rule when `_filter` is null or equals the rule's `type`. Colour: free green, paid `Color(0xFF1E6FD9)`, no red. Icon (white, size 18): free `Icons.money_off`, paid `Icons.attach_money`, no `Icons.block`. Tapping opens the rule sheet.
3. **Garage markers** (80 by 60), only when `_filter == 'garage'`. A navy `Color(0xFF0A4296)` square with a white bold "P" (size 18), and below it the text `$price/hr` in navy, size 12, weight 600.
4. **Open-spot markers** (40 by 40 navy circles with a white `Icons.local_parking`), shown unless the filter is `garage`, and only where `taken != true`. Tapping opens the spot sheet.
5. **Destination marker**: `Icons.place`, red, size 40, only when a destination is set.

Each marker layer is its own `StreamBuilder` wrapping a `MarkerLayer` through one helper function, so the map never rebuilds all streams at once.

8.4 Top overlay

Positioned below the status bar (safe-area top plus 8), 12 px side margins, as a column:

1. A row with two cards at opposite ends. Left: a card with the text "ParkPulse" (weight 800, size 16). Right: a card with an orange `Icons.monetization_on` coin and the user's points (bold, size 16), live from a stream on `users/{uid}`, showing 0 when empty.
2. An 8 px gap, then a search field with elevation 2 and rounded 16 px corners: hint "Search address or place", a search icon on the left, a close icon on the right that clears the text, no border. Submitting calls the Nominatim search (`format=json&limit=1`), moves the map to the first result at zoom 16, and shows "Place not found" or "Search failed. Check your connection." as needed.

3. An 8 px gap, then a horizontally scrollable row of three `FilterChip`s with no checkmark: "Free" (`Icons.money_off`), "Paid" (`Icons.attach_money`), "Garages" (`Icons.local_parking`). Selecting a chip sets `_filter` ; deselecting sets it to null. Only one is active at a time.

8.5 Bottom overlay

Positioned at the bottom (safe-area bottom plus 12), 12 px side margins, a column with a 12 px gap between its two parts:

- **Row, bottom-aligned.** On the left, a tonal filled button with a bell icon and the label "Get open spot alerts" (changes to a close icon while picking, and the label becomes "Alerts on" once a destination exists). Pressing it toggles destination-picking and shows the message "Tap the map to set your destination". On the right, a column aligned to the right with 10 px gaps: a small round "my location" button (recentres the map at zoom 16), an extended red button (`Colors.red.shade600` , white content, `Icons.place`) labelled "Report Spot", and an extended blue button (`Color(0xFF1E6FD9)` , white content, `Icons.local_parking`) labelled "Report Rule".
- **A full-width-ish filled button** (colour `Color(0xFF1F1FA8)` , minimum size 260 by 52, `Icons.document_scanner` , label "Scan Parking Sign" at size 16), centred.

Give every floating button a unique `heroTag` .

8.6 Report Spot flow

Get the GPS location (if permission is denied, show the error text in a snackbar and stop). Look up the city with Nominatim reverse geocoding (`zoom=10` , use `city` , then `town` , `village` , `county` , else empty). Open a modal bottom sheet (scrollable above the keyboard) titled "Report parking spot" (bold, size 20) containing, in order: a `SegmentedButton` "Street" / "Garage" (default Street); a text field with an outline border, label "Optional note (e.g. opposite the cafe)" for Street or "Garage name" for Garage; for Garage only, a numeric field labelled "Price per hour"; grey text "City: <city>" or "City not detected"; a full-width filled button "Save spot". Saving closes the sheet. Street writes to `spots` with `lat` , `lng` , `uid` , `city` , `note` , `taken: false` , `createdAt: FieldValue.serverTimestamp()` . Garage writes to `garages` with `lat` , `lng` , `uid` , `city` , `name` , `price` (0 if empty), `createdAt` . Then show "Shared. Thanks!". The app never writes points; Cloud Functions do.

8.7 Rule dialog

Signature `_addRule(LatLng ll, {String text = '', String type0 = 'paid'})`. An `AlertDialog` titled "Add a sign or rule here" with a dropdown of "Free parking" (`free`), "Paid or time limit" (`paid`), "No parking" (`no`), and a text field (max 150 characters, hint "2 HR, 8AM-6PM, Mon-Fri") pre-filled with `text`. Buttons "Cancel" and "Save". Do nothing if cancelled or the text is empty. Otherwise write to `rules` and show "Rule added. +5 points". "Report Rule" uses the current GPS location; a long-press uses the pressed point.

8.8 Sheets

- **Spot sheet:** title "Open spot" (bold, size 20), then "Reported N min ago" (from `createdAt`, 0 if still null), then a filled button "I took it (+3 points)" that closes the sheet and updates the document with `taken: true` and `takenBy: uid`.
- **Rule sheet:** padding 20, the rule `text` at size 18.

8.9 Alerts

Pressing the destination button then tapping the map sets `_dest`, writes `alerts/{uid}` with `alertOn: true`, `destLat`, `destLng` (merge), and shows "Alerts on. We will notify you even when the app is closed." On start, request notification permission, get the FCM token, save it to `alerts/{uid}` (merge) and listen to token refresh. In the app, also listen to the spots stream: skip the first snapshot, and for each newly added spot from another user within 500 m of the destination (use `Distance().as(LengthUnit.Meter, ...)`) show the snackbar "A spot just opened N m from your destination". Closed-app alerts come from the Cloud Function.

8.10 Scan Parking Sign

Open the camera with `ImagePicker` (max width 1280). Get the GPS location. Show "Reading the sign...". POST the image as multipart field `file` to `$kSignApi/detect`. Parse `signs`; if empty show "No sign found. Try a closer photo." Otherwise take the first sign and open the rule dialog at the user's location with text = `rule.summary` if a rule exists, else the raw OCR `text`, and type = `no` when `rule.kind == 'no'` else `paid`. The user can edit the text before saving. Errors show "Could not read the sign: <error>".

8.11 Location helper

Check and request permission with Geolocator. If denied (or denied forever) throw the text "Allow location access to use this." Otherwise return the current position as a `LatLng`.

9. Sign reader server (`sign_ai/`)

`server.py`: FastAPI app with one route `POST /detect` taking an uploaded file. Build an `easyocr.Reader(['en'], gpu=False)` once at start. If the file at env `SIGN_WEIGHTS` (default `runs/detect/signs/weights/best.pt`) exists, load it with `ultralytics.YOLO`. For each request: open the image as RGB; if a model is loaded, predict with `conf=0.35`, crop each box, and keep its class name and confidence; if there are no crops (no model, or nothing found) use the whole image with class `whole_image` and confidence 1.0. OCR each crop (`detail=0`, joined with spaces). Return `{ "signs": [{"class", "confidence", "text", "rule"}], "used_detector": bool }`, where `rule` is the decoder output or null.

`train.py`: `YOLO('yolov8n.pt').train(data='data.yaml', epochs=60, imgsz=640, batch=16, name='signs')`.

`data.yaml`: path `dataset`, train `images/train`, val `images/val`, classes 0 `no_parking`, 1 `time_limit`, 2 `paid_meter`, 3 `permit_only`, 4 `other_sign`.

`sign_ai/README.md`: explain collecting 300 to 500 sign photos, labelling in Roboflow or CVAT in YOLOv8 format with an 80/20 split, training on a free Google Colab GPU, copying `best.pt` next to the server, aiming for mAP50 above 0.8, and deploying over HTTPS (Cloud Run, Render or a VPS).

10. Decoder (`sign_ai/decode.py`)

`decode(raw, now=None)` uppercases the text, collapses whitespace, returns `None` if shorter than 4 characters, and returns a dict `{kind, summary, verdict}` or `None`.

- **Days:** default every day. A range like `MON-FRI` or `MON TO FRI` expands (wrapping past Sunday). `EXCEPT SUN` removes that day. Output text: "every day", "Monday to Friday" (exactly Mon to Fri), else a comma-separated list of full day names.
- **Times:** find every window like `8AM-6PM` or `7-9AM` (the first meridiem may be missing; use the second). Multiple windows are joined with " and " and formatted like "8 AM to 6 PM". No window means "at all times".

- **Active now:** today is in the day list and the current minute falls inside any window. Python weekday is converted so Sunday is 0.
- **Limit:** `N HR`, `N HOUR` or `N MIN` gives "1 hour", "2 hours" or "30 minutes".
- **Order of checks:** `NO PARKING/STOPPING/STANDING` gives kind `no`; then a time limit gives kind `limit`; then `PERMIT` gives kind `no`; then `METER/PAY/PAID` gives kind `limit`; otherwise `None`.
- **Wording:** no parking: summary "No parking <times>, <days>."; verdict "Don't park here now." when active, else "You can park now. The rule isn't active." Limit: summary "Up to <limit>, <times>, <days>. No limit outside those times."; verdict "You can park now, but move within <limit>." when active, else "You can park now with no limit." Permit: "Permit holders only, <times>, <days>." with "Skip this street unless you hold a permit." or "Permit rule isn't active now." Paid: "Paid parking <times>, <days>." with "Pay before you leave the car." or "Free right now."

Tests it must pass (Monday 26 Oct 2026):

- 10:00, `2 HR PARKING 8AM-6PM MON-FRI EXCEPT SUN` gives kind `limit`, summary "Up to 2 hours, 8 AM to 6 PM, Monday to Friday. No limit outside those times.", verdict "You can park now, but move within 2 hours."
- 10:00, `NO PARKING 7-9AM 4-6PM MON-FRI` gives kind `no`, summary "No parking 7 AM to 9 AM and 4 PM to 6 PM, Monday to Friday.", verdict "You can park now. The rule isn't active."
- 17:00 with the same sign gives verdict "Don't park here now."
- Sunday 10:00 with the first sign: the Sunday exclusion means the 2-hour rule is not active.

11. Quality rules

- Never create streams inside `build`.
- Check `mounted` before using `context` after an `await`.
- Use real, compiling Dart; no pseudo-code.
- Show friendly messages in snackbars for every failure.
- Keep everything in one `main.dart` for this prototype.
- Do not hard-code secrets.

12. Acceptance tests

1. App opens, signs in anonymously, centres on the user's GPS.
2. Report a street spot on phone A; the navy P pin appears on phone B within seconds, and A's points rise by 10 (via the function).
3. Phone B sets a destination within 500 m of the spot; phone A reports another spot; B gets a push notification even with the app closed.
4. Tapping "I took it" removes the pin for everyone and gives B +3 points.
5. Long-press adds a rule pin; chip filters show only matching pins; the Garages chip shows price labels.
6. Scanning a clear printed sign opens the rule dialog pre-filled with plain English.
7. Pins older than 30 minutes disappear.

13. Known limits to state honestly in the README

No street-segment colour lines (needs city open data), no photo upload on reports, no spam rate limiting, OpenStreetMap tiles and Nominatim are for light use only, OCR works best on a close, straight photo until a YOLOv8 model is trained.

14. Demo script (for the presenter)

1. Show the map and the three filter chips.
2. On phone A report a spot; watch it appear on phone B.
3. On B set a destination, then report another spot from A and show the push alert.
4. Scan a printed sign and show the plain-English result.
5. Show the points chip going up. Finish with the roadmap: trained sign detector, city open data, rewards for premium features.